

Introduction to Python

Topics Discussed

- What is Python?
- Basics of Python Programming
- Control structures in Python
- Functions in Python
- Modules in Python

What is Python?

- Python is a general purpose programming language that is often applied in scripting roles
- So, Python is programming language as well as scripting language.
- Python is also called as Interpreted language
- Developed in 1991 by Guido van Rossum.
- Intuitive and minimal coding is required
- Dynamically typed, type declarations is not required, data type is tracked at runtime
- Automatic memory management
- Spacing(Indentation) defines blocks of code such as control structures and blocks

```
a=5
print(a)
print(id(a))
b=5
print(b)
print(id(b))
a=5.77
print(a)
print(id(a))

5
9793216
5
9793216
5.77
139882956520976
```

Why to Learn Python?

Advantages

Open Source

- Freely available, can know everything by analyzing its code

Popular

- Great humanity support when you run in problems

Accessible

- From Budding Programmer to a Professional One

Everywhere

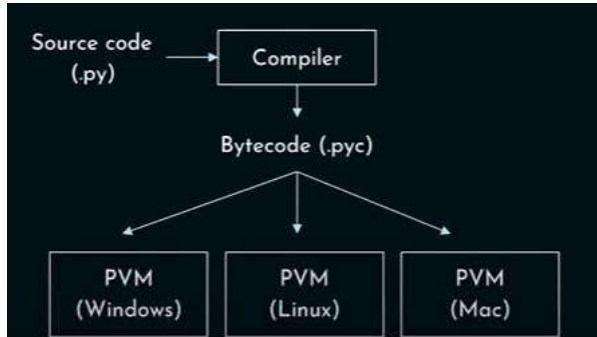
- Used almost everywhere throughout the world

Fun

- It is expressive, readable style, quick editing, run and development environment.
- Ease of programming
- Minimal time to develop and maintain codes
- Modular and object-oriented
- Great standard and user-contributed libraries
- Large user groups

Disadvantages

- Interpreted and hence slower than compiled languages
- Not great for 3D graphic applications requiring intensive computations



Python is Hybrid Language(Compiled and Interpreted)

- To see the bytecode of program
- `import py_compile`
- `py_compile.compile('hello.py') 'pycache\hello.cpython-310.pyc'`

Jupyter Notebook

- Web interface to Python, introduced in 2015
- Rich text, improved graphical capabilities
- Integrate many existing web libraries for data visualization
- Allow to create and share documents that contain live code, equations, visualizations and explanatory text.
- Interface with over 40 languages, such as R, Julia and Scala
- This colab.research.google.com notebook is a Jupyter notebook [back to top](#)

Variables

- Variable type in Python is dynamically determined from the value it is assigned, no data type is declared
- Assign meaningful variable names
- Some keywords are reserved such as 'assert', 'while', 'lambda', 'for', 'if', etc

List of Keywords in Python

- The following identifiers are used as reserved words, or keywords of the language, and cannot be used as ordinary identifiers. They must be spelled exactly as written here:

False	await	else	import	pass
None	break	except	in	raise
True	class	finally	is	return
and	continue	for	lambda	try
as	def	from	nonlocal	while
assert	del	global	not	with
async	elif	if	or	yield

- A variable is assigned a value using the '=' operator, example: a = 5

```
a = 5 #Variable declaration and initialization
print(type(a))
b=8.9
print(type(b))
```

```
<class 'int'>
<class 'float'>
```

id(a) #Each object is assigned an unique identity. id() is used to fetch this integer value

```
9793216
```

```
A = 99.9
print(type(A))
print(id(A))
```

```
<class 'float'>
135535094049424
```

```
Name = 'Symbiosis'
print(type(Name))
len(Name)
```

```
<class 'str'>
9
```

```
Name = Name.upper() #upper() is used to convert string to upper case
print(Name)
print(Name.lower()) #lower() is used to convert string to lower case
Name
```

```
SYMBIOSIS
symbiosis
'SYMBIOSIS'
```

```
print(a, A, Name)
```

```
5 99.9 SYMBIOSIS
```

```
a, A, Name = 10, 3.14, 'Python'
print(a, Name, A)
```

```
10 Python 3.14
```

```
x = y = z = 11
print(x, y, z)
```

```
11 11 11
```

```
print(isinstance(a, int))
print(isinstance(A, float))
print(isinstance(Name, int))
```

```
True
True
False
```

```
x = str(3) #type casting in python, str() constructor is used to convert 3 to string
print(type(x))
y = 3
print(type(y))
y = float(3) #type casting in python, float() constructor is used to convert 3 to float
print(type(y))
```

```
<class 'str'>
<class 'int'>
<class 'float'>
```

Built-in Data Types in Python

Text Types	str
Numeric Types	int, float, complex
Sequence Types	list, tuples, range
Mapping Types	dict
Set Types	set, frozenset
Boolean Types	bool
Binary Types	bytes, bytearray, memoryview

Types of Literals

Numerical Literals

```
#Integer Literals
a = 0b100 #Binary Literal
b = 99 #Decimal Literal
c = 0o170 #Octal Literal
d = 0x10a #Hexadecimal Literal
print(a, b, c, d)
print(int("322", 5)) #Converting 322 in base 5 number system to integer in decimal
print(bin(112))
```

```
4 99 120 266
87
0b1110000
```

```
#Complex Literals
h = 2 + 3.14j
print(h)
complex(3, 2)
```

```
(2+3.14j)
(3+2j)
```

String Literals

```
Name = "Symbiosis"
char = "S"
Address_Mul = ""57, Mahesh Nagar,
Behind Anant Nagar, Nagpur - 440013""
Str_raw = r"This is \n raw string \u00dc" #raw string treats backslash (\) as a literal character
Str_uni = u"\u00dc nic\u0f06 de"
print(Name, char, Address_Mul, Str_raw)
print(Str_uni)
```

```
Symbiosis S 57, Mahesh Nagar,
Behind Anant Nagar, Nagpur - 440013 This is \n raw string \u00dc
Ü nic de
```

```
print(ord("A"))
print(ord("Ü"))
```

```
65
220
```

```
print(int(0x00dc))
```

```
220
```

```
chr(100)
```

```
'd'
```

Boolean Literals

```
x = (1 == True)
y = (1 == False)
a = True + 4
b = False + 10
print(x, y, a, b)
```

```
True False 5 10
```

```
a = int(input('Give first number'))
b = int(input('Give second number'))
c = (a*b)
chr(c)
```

```
Give first number75
Give second number32
'᳚'
```

```
abs(3-5)
```

```
2
```

```
a = int(input('Give first number'))
b = int(input('Give second number'))
c=a+b
d=c%2
e=(d==0)
print(e)
print((a+b)%2==0)
```

```
Give first number8
Give second number6
True
True
```

```
a=float(input("Enter the 1st value"))
b=float(input("Enter the 2nd value"))
c=(a*b)
print(int(c))
```

```
Enter the 1st value2.2
Enter the 2nd value2.2
4
```

```
name=input("Enter Name: ")
colg=input("Enter Institute Name: ")
Add=input("Enter Institute Address: ")
Sem=int(input("Enter Semester: "))
print("\n",name,"\n",colg,"\n",Add,"\n",Sem)
```

```
Enter Name: Riya
Enter Institute Name: SIT, Nagpur
Enter Institute Address: Wathoda
Enter Semester: 2
```

```
Riya
SIT, Nagpur
Wathoda
2
```

```
#special literals
val=None
print(val)
```

```
None
```

Strings

```

my_str = "sitnagpur"

print(my_str); len(my_str)

    sitnagpur
    9

#indexing starting at 0
print(my_str[2]); print(my_str[1]); print(my_str[6]); print(my_str[-1])

    t
    i
    p
    r

#string slicing
print(my_str[::-2])
print(my_str[1:4])
print(my_str[:])
print(my_str[1:])
print(my_str[:4])
print(my_str[-4:-1])

    rpat
    itn
    sitnagpur
    itnagpur
    sitn
    gpu

my_str[::-1]

    'sitnagpur'

m = '0123456789'
m
print(m[1:4:1])#start point:stop point:No of displacement steps
print(12//5) #floor division-rounds the resultant value down to the nearest integer

    123
    2

#string split
my_str = "SIT Nagpur Wathoda"
s1 = my_str.split()
my_str1 = "Python#Programming#Class"
s2,s3,s4=my_str1.split(sep='#')
print(s1)
print(s2)
print(s3, s4)

    ['SIT', 'Nagpur', 'Wathoda']
    Python
    Programming Class

name=input("enter your name")
a1,a2,a3=name.split()
print("first name",a1);
print("middle name",a2);
print("last name",a3);

    enter your nameRiya Rajesh Patil
    first name Riya
    middle name Rajesh
    last name Patil

my_str = "Symbiosis International"
s1,s2 = my_str.split();
print(s1, type(s1))

    Symbiosis <class 'str'>

```

```
#multiple line spanning
multilines= """This is a multi-line
block of sample
text"""
print(multilines)
```

```
This is a multi-line
block of sample
text
```

```
#string formatting
Name = input("Enter a name: ")
print("Hello",Name.upper());
print(type(Name))
```

```
Enter a name: Riya
Hello RIYA
<class 'str'>
```

```
a=int(input("Enter 1st Number: "))
b = int(input("Enter 2nd Number: "))
a+b
x, y = input("Enter two Numbers: ").split()
int(x)+int(y)
```

```
Enter 1st Number: 7
Enter 2nd Number: 8
Enter two Numbers: 67 89
156
```

```
p,r,n,t = input("Give values of p, r, n, t ").split()
print('Compound interest is', int(p)*(1+(int(r)/ int(n))**(int(t))))
```

```
Give values of p, r, n, t 1 2 3 4
Compound interest is 1.1975308641975309
```

```
a=float(input("num1"))
b=float(input("num2"))
a*b
```

```
num145.65
num265.89
3007.8785
```

```
Name1 = "Manmeet"
number_of_bread = 300
text = "%d bread in the fridge, %s and %s will eat" %(number_of_bread,a1, Name1)
print(text)
```

```
300 bread in the fridge, Riya and Manmeet will eat
```

```
type(a1)
```

```
str
```

```
total= "my share = %.5f, tip = %5d" %(24.5, 555.5)
print(total)
```

```
my share = 24.50000, tip = 555
```

```
#more string ops
#https://docs.python.org/3/library/stdtypes.html#string-methods
my_str="Hello World"
print(my_str.upper())
print(my_str.isalpha()) #returns True if all the characters are alphabet letters (a-z)
print(my_str)
print(my_str[0].isalpha())
a='12'
print(a.isdigit())
print(my_str.capitalize()) #convert the first character of a string to uppercase and the remaining characters to lowercase
print(my_str.isnumeric()) #returns True if all the characters are numeric (0-9), otherwise False
```

```

HELLO WORLD
False
Hello World
True
True
Hello world
False

my_str="PRITAM"
prn= "22070521134"
base="0.2"
print(base.isnumeric())
print(prn.isnumeric())
print(prn.isdecimal())

False
True
True

#more string ops
#https://docs.python.org/3/library/stdtypes.html#string-methods
my_str='PYTHON123'
print("check if all char are alphanumeric which is combination of alpabets and numbers")
print(my_str.isalnum()) #check if all char are alphanumeric numbers
print("check if all char in the string are alphabetic")
print(my_str.isalpha()) #check if all char in the string are alphabetic
print("test if string contains title words")
print(my_str.istitle()) #returns True if all words in a text start with a upper case letter, AND the rest of the word are lower case letters
print("test if string contains upper case")
print(my_str.isupper()) #test if string contains upper case
print("test if string contains lower case")
print(my_str.islower()) #test if string contains lower case
print("test if string contains spaces")
print(my_str.isspace()) #test if string contains spaces
print("if string endswith 3")
print(my_str.endswith('3')) #test if string endswith a d
print("test if string starts with P this is casesensitive")
print(my_str.startswith('P')) #test if string startswith P

check if all char are alphanumeric which is combination of alpabets and numbers
True
check if all char in the string are alphabetic
False
test if string contains title words
False
test if string contains upper case
True
test if string contains lower case
False
test if string contains spaces
False
if string endswith 3
True
test if string starts with P this is casesensitive
True

```

Lists

- Collection of data []
- Often used to store homogeneous values
 - e.g., Numbers, names with the same data type
- Mutable: modify in place without creating a new object

```

my_list = [1,2,9,4]
print(my_list)
#how to find out the data type?

```

```
[1, 2, 9, 4]
```

```
print(type(my_list))
```

```
<class 'list'>
```

```

my_list2 = list(range(-4,10,2))
print(my_list2)

[-4, -2, 0, 2, 4, 6, 8]

# access list members by index
# start at 0, end at n-1
print("1st element =", my_list[0])
print("last element =", my_list[3])

1st element = 1
last element = 4

#Contatenate lists
print(my_list + my_list2)
print(my_list *2)
print(my_list *3)
print(my_list *5)
print(my_list)

[1, 2, 9, 4, -4, -2, 0, 2, 4, 6, 8]
[1, 2, 9, 4, 1, 2, 9, 4]
[1, 2, 9, 4, 1, 2, 9, 4, 1, 2, 9, 4]
[1, 2, 9, 4, 1, 2, 9, 4, 1, 2, 9, 4, 1, 2, 9, 4, 1, 2, 9, 4]
[1, 2, 9, 4]

# modify list
del my_list[0]; print(my_list)
my_list.append(99); print(my_list)
my_list.insert(1, 100); print(my_list)

[2, 9, 4]
[2, 9, 4, 99]
[2, 100, 9, 4, 99]

#some list funs
my_list.sort(); print(my_list)
my_list.reverse(); print(my_list)

[2, 4, 9, 99, 100]
[100, 99, 9, 4, 2]

my_list4 = ['1','70','21','100']
y = my_list4.sort()
print(my_list4)
#print(y)

['1', '100', '21', '70']

#more sorting
my_list3 = ['cats', 'SHEEP','DOGS','dogs','horses']
print(sorted(my_list3))

print(sorted(my_list3, reverse=True))
print(my_list3)

['DOGS', 'SHEEP', 'cats', 'dogs', 'horses']
['horses', 'dogs', 'cats', 'SHEEP', 'DOGS']
['cats', 'SHEEP', 'DOGS', 'dogs', 'horses']

print('{}'.format(my_list3.sort()))

None

# sort by lowercase
sorted(my_list3, key=str.lower)

['cats', 'DOGS', 'horses', 'SHEEP']

```

Tuples

- Why Tuple?
- Processed faster than lists
- Sequence of a Tuple is protected
- Sequence unpacking

Tuple is same as **List**, the only difference is that tuples are **immutable**. **Faster** than list as it cannot be modified dynamically

Suprising Trait:

Python tuples are immutable, but their values may change. This may happen when a tuple holds a reference to any mutable object, such as a list. ... It's clear that dum and dee refer to objects that are equal, but not to the same object.

```
my_tup=(1, 'two', 3)
print(my_tup)
my_tuple = 1, 2, 'three'
print(my_tuple)
#empty tuples
my_empty_tuple = ()
print(my_empty_tuple)
```

```
(1, 'two', 3)
(1, 2, 'three')
()
```

you have to include a comma, even though there is only one value

```
tup1 = (50,);
print(tup1)
type(tup1)
```

```
(50,)
tuple
```

```
tup2 = (50, 23)
print(type(tup2))
tup2 = 50,
type(tup2)
```

```
<class 'tuple'>
tuple
```

```
tup2[1]=32
```

```
-----
TypeError                                Traceback (most recent call last)
<ipython-input-8-f5f936d5c5d7> in <module>
----> 1 tup2[1]=32
```

TypeError: 'tuple' object does not support item assignment

SEARCH STACK OVERFLOW

```
my_tup = (2, 8, 3, 1 )
print(my_tup)
a,b,c,d = my_tup
print(a, b, c, d)
```

```
(2, 8, 3, 1)
2 8 3 1
```

```
#print(my_tup[0])
```

```
names=['Mr. Patil','Mrs. JAdhav','MRS']
list(filter(lambda x: x.upper().split('.')[0]=='MRS', names))
```

```
['Mrs. JAdhav', 'MRS']
```

```
tup = ('physics', 'chemistry', 1997, 2000 )
print(tup)
```

```
('physics', 'chemistry', 1997, 2000)
```

```
del tup
print("After deleting tup : ");
print(tup)
```

After deleting tup :

```
-----
NameError                                Traceback (most recent call last)
<ipython-input-60-8f9bcd963457> in <module>
      1 del tup
      2 print("After deleting tup : ");
----> 3 print(tup)

NameError: name 'tup' is not defined
```

SEARCH STACK OVERFLOW

Immutable? Not?

```
dum = ('1861-10-23', ['poetry', 'pretend-fight'])
dee = ('1861-10-23', ['poetry', 'pretend-fight'])
print(dum == dee)
print(dum is dee)
```

```
id(dum), id(dee)
```

```
True
False
(139969034892800, 139969034939968)
```

```
t_doom = dum
print(dum, t_doom)
```

```
print(t_doom == dum)
print(t_doom is dum)
id(t_doom), id(dum)
```

```
('1861-10-23', ['poetry', 'pretend-fight']) ('1861-10-23', ['poetry', 'pretend-fight'])
True
True
(140667638476864, 140667638476864)
```

```
skills = t_doom[1]
print(skills)
```

```
skills.append('rap')
print(t_doom)
```

```
['poetry', 'pretend-fight']
('1861-10-23', ['poetry', 'pretend-fight', 'rap'])
```

```
print(dum == dee)
print(dum)
```

```
False
('1861-10-23', ['poetry', 'xyz', 'pretend-fight', 'rap'])
```

```
dum[1]=[1,2,3]
```

```
-----
TypeError                                Traceback (most recent call last)
<ipython-input-6-83d7dd421106> in <module>
----> 1 dum[1]=[1,2,3]

TypeError: 'tuple' object does not support item assignment
```

SEARCH STACK OVERFLOW